disco

Density-based Internal Score for Clustering Outcomes

R Translation, Implementation & Experimental Validation

Author: Amin Entezari

amin_entezari@outlook.com

Reference implementation: Beer, Krieger, Weber, Ritzert, Assent, Plant (2025)

DISCO: Internal Evaluation of Density-Based Clustering

arXiv:2503.00127

GitHub: `https://github.com/aminentezari/Disco-R`

R Package: `devtools::install_github("aminentezari/Disco-R")`

April 20, 2026

This document describes the **faithful R translation** of the Python DISCO reference implementation, the three implementation bugs discovered and resolved, and numerical verification across **eight** benchmark datasets including both real-world and synthetically generated data.

Contents

1 Introduction **2**

2 Algorithm **2**
2.1 Problem Statement . . . 2
2.2 Step 1: Core Distance . . . 2
2.3 Step 2: Mutual Reachability Distance . . . 2
2.4 Step 3: DC-Distance via Minimum Spanning Tree . . . 2
2.5 Step 4: Scoring Cluster Points . . . 3
2.6 Step 5: Scoring Noise Points . . . 3
2.7 Overall Score . . . 3

3 R Implementation **3**
3.1 Translation Approach . . . 3
3.2 Installation and Quick Start . . . 3

4 Implementation Bugs Discovered and Fixed **3**
4.1 Bug 1 — Core Distance Off-by-One . . . 3
4.2 Bug 2 — MST Tie-Breaking Difference . . . 4
4.3 Bug 3 — K-Means Cross-Language Non-Determinism . . . 4
4.4 Summary of Bugs . . . 4

5 Experimental Validation **5**
5.1 Dataset Overview and Hyperparameters . . . 5
5.2 Clustering Hyperparameters per Dataset . . . 5
5.3 Generated Dataset Parameters . . . 6
5.4 Numerical Results . . . 6
5.5 Interpretation of Differences . . . 7
5.6 Cross-Dataset Summary . . . 7

6 API Reference **8**
6.1 `disco_score(X, labels, min_points = 5)` . . . 8
6.2 `disco_samples(X, labels, min_points = 5)` . . . 8
6.3 `compute_dc_distances(X, min_points = 5)` . . . 8
6.4 Utility Functions . . . 9

7 Score Interpretation Guide **9**

8 Conclusion **9**

1. Introduction

DISCO (Density-based Internal Score for Clustering Outcomes) is a cluster validity index (CVI) introduced by Beer et al. [**?**] that evaluates the quality of density-based clustering results *without requiring ground truth labels.*

Unlike classical CVIs such as the Silhouette Coefficient [**?**] and Davies-Bouldin Index, DISCO:

- Handles **arbitrary-shaped clusters** by replacing Euclidean distance with *density-connectivity distance* (dc-distance);
- Evaluates **noise point quality** explicitly via two sub-scores (p_{sparse} and p_{far});
- Returns a bounded, interpretable score in $[-1, 1]$.

This report documents:

1. The mathematical foundations and algorithmic pipeline (§2);
2. The R implementation and its translation from the Python reference (§3);
3. Three implementation bugs discovered and corrected (§4);
4. Numerical verification across eight benchmark datasets (§5);
5. The public API reference (§6).

2. Algorithm

2.1 Problem Statement

Given a dataset $\mathbf{X} = \{x_1, \ldots, x_n\} \subset \mathbb{R}^p$ and a clustering $\mathcal{C} = \{C_1, \ldots, C_K\}$ with optional noise set $\mathcal{N} = \{x_i : \ell_i = -1\}$, DISCO assigns a quality score $\rho(x_i) \in [-1, 1]$ to every point and returns the mean as the overall score.

2.2 Step 1: Core Distance

The *core distance* of a point x_i with parameter μ (min_points) is the distance to its $(\mu - 1)$-th nearest neighbour:

$$\kappa_\mu(x_i) = d\big(x_i,\, x_i^{(\mu-1)}\big)$$

2.3 Step 2: Mutual Reachability Distance

$$d_{\text{reach}}(x_i, x_j) = \max\big(\kappa_\mu(x_i),\ \kappa_\mu(x_j),\ d(x_i, x_j)\big)$$

2.4 Step 3: DC-Distance via Minimum Spanning Tree

1. Construct the complete graph G with edge weights d_{reach}.
2. Compute the Minimum Spanning Tree T of G using Prim's algorithm.
3. The **dc-distance** between x_i and x_j is the maximum edge weight on the unique path between them in T:

$$d_{\text{DC}}(x_i, x_j) = \max_{e \in \text{path}_T(i,j)} w(e)$$

2.5 Step 4: Scoring Cluster Points

$$a(i) = \frac{1}{|C_{c_i}| - 1} \sum_{j \in C_{c_i}, j \neq i} d_{\mathrm{DC}}(x_i, x_j) \tag{1}$$

$$b(i) = \min_{c \neq c_i} \frac{1}{|C_c|} \sum_{j \in C_c} d_{\mathrm{DC}}(x_i, x_j) \tag{2}$$

$$\rho_{\mathrm{cluster}}(x_i) = \frac{b(i) - a(i)}{\max(a(i), b(i))} \tag{3}$$

2.6 Step 5: Scoring Noise Points

$$p_{\mathrm{sparse}}(x_n) = \min_c \frac{\kappa_\mu(x_n) - M_c}{\max(\kappa_\mu(x_n),\ M_c)}, \qquad p_{\mathrm{far}}(x_n) = \min_c \frac{d_{\min,c}(x_n) - M_c}{\max(d_{\min,c}(x_n),\ M_c)}$$

$$\rho_{\mathrm{noise}}(x_n) = \min\big(p_{\mathrm{sparse}}(x_n),\ p_{\mathrm{far}}(x_n)\big)$$

2.7 Overall Score

$$\mathrm{DISCO}(\mathbf{X}, \mathcal{C}) = \frac{1}{n} \sum_{i=1}^{n} \rho(x_i)$$

3. R Implementation

3.1 Translation Approach

The R package `disco` is a **line-by-line translation** of the Python reference implementation (`disco.py` + `dctree.py`). The package consists of a single file `R/disco.R` combining three logical sections: DC-Distance infrastructure (from `dctree.py`), DISCO scoring functions (from `disco.py`), and utility functions.

3.2 Installation and Quick Start

```
devtools::install_github("aminentezari/Disco-R")
library(disco)
library(dbscan)

data      <- make_circles(n_samples = 300, noise = 0.05,
                          factor = 0.4, random_state = 42)
X         <- data$X
db_labels <- dbscan(X, eps = 0.2, minPts = 5)$cluster - 1L
km_labels <- kmeans(X, centers = 2, nstart = 10)$cluster - 1L

disco_score(X, db_labels)   # 0.6900921813
disco_score(X, km_labels)   # 0.0270091908
```

4. Implementation Bugs Discovered and Fixed

4.1 Bug 1 — Core Distance Off-by-One

Root Cause

Python's `np.partition(eucl_dists[i], k)[:k]` **includes self (dist=0)**, so the effective core distance is the $(k-1)$-th nearest neighbour. `FNN::get.knn` **excludes self**, requiring `k = min_points - 1`.

Fix

```
k           <- min_points - 1
knn_result  <- FNN::get.knn(X, k = k)
core_dists  <- knn_result$nn.dist[, k]
```

4.2 Bug 2 — MST Tie-Breaking Difference

Root Cause

`igraph::mst` breaks equal-weight edges differently than Python's custom Prim (`dctree.py`, lines 401–438), producing different MSTs and therefore different dc-distances.

Fix

Replaced `igraph::mst` with an exact R port of Python's `_get_mst_edges()`.

4.3 Bug 3 — K-Means Cross-Language Non-Determinism

Root Cause

`set.seed(42)` in R and `random_state=42` in Python use different RNGs, producing different K-Means partitions.

Fix

```
pd.DataFrame({"km_label": km.labels_}).to_csv("python_km_labels.csv
    ",
                                                index=False)
```

```
km_labels <- as.integer(read.csv("python_km_labels.csv")$km_label)
```

4.4 Summary of Bugs

#	Location	Root Cause	Fix
1	`calculate_reachability_distance`, `p_noise`	Python includes self in k-NN; FNN excludes self	`k = min_points - 1`
2	`get_mst_edges`	`igraph::mst` tie-breaking differs from Python's Prim	Port Python's exact `_get_mst_edges()`
3	All experiment scripts	R and Python RNGs produce different K-Means partitions	Export Python labels as CSV; load in R

5. Experimental Validation

Eight datasets were used to validate the R implementation against the Python reference. All DBSCAN and K-Means labels are generated in Python first and loaded in R to guarantee identical partitions.

5.1 Dataset Overview and Hyperparameters

Dataset	Source	n	K	Shape	DBSCAN eps	DISCO μ
Concentric Circles	Generated	300	2	Rings	0.20	5
Two Moons	Generated	300	2	Crescents	0.20	5
Complex9	`complex9.arff`	3031	9	Mixed	15.0	5
Complex8	`complex8.arff`	2551	8	Mixed	10.0	5
Dartboard1	`dartboard1.arff`	1000	4	Rings	0.08	5
3-Spiral	`3-spiral.arff`	312	3	Spirals	2.0	5
Blobs	Generated	600	3	Gaussian blobs	0.5	5
Diagonal Blobs	Generated	600	3	Elongated diagonal	0.8	5

Notes: K = number of clusters for K-Means; DBSCAN `min_samples` = 5 for all datasets except 3-Spiral (`min_samples` = 3). DISCO μ = `min_points` parameter = 5 throughout.

5.2 Clustering Hyperparameters per Dataset

Dataset	DBSCAN Parameters	K-Means Parameters
Concentric Circles	`eps=0.2, min_samples=5`	`k=2, random_state=42, n_init=10`
Two Moons	`eps=0.2, min_samples=5`	`k=2, random_state=42, n_init=10`
Complex9	`eps=15.0, min_samples=5`	`k=9, random_state=42, n_init=10`
Complex8	`eps=10.0, min_samples=5`	`k=8, random_state=42, n_init=10`
Dartboard1	`eps=0.08, min_samples=5`	`k=4, random_state=42, n_init=10`
3-Spiral	`eps=2.0, min_samples=3`	`k=3, random_state=42, n_init=10`
Blobs	`eps=0.5, min_samples=5`	`k=3, random_state=42, n_init=10`
Diagonal Blobs	`eps=0.8, min_samples=5`	`k=3, random_state=42, n_init=10`

5.3 Generated Dataset Parameters

Dataset	Generation Parameters
Concentric Circles	`make_circles(n=300, noise=0.05, factor=0.4, seed=42)`
Two Moons	`make_moons(n=300, noise=0.05, seed=42)`
Blobs	`MASS::mvrnorm`, 3 centres: $(1.5, 7.0)$, $(6.5, 7.0)$, $(4.0, 2.5)$; $\sigma^2 = 0.55^2\mathbf{I}$; `seed=42`; 150 pts per cluster
Diagonal Blobs	`MASS::mvrnorm`, 3 centres: $(2.5, 5.0)$, $(0.5, 1.8)$, $(-1.5, -1.5)$; $\Sigma = \left[\begin{smallmatrix} 1.2 & -1.0 \\ -1.0 & 1.2 \end{smallmatrix}\right]$; `seed=42`; 200 pts per cluster

5.4 Numerical Results

All results shown to 20 decimal places. Diverging digits are highlighted in red (R) and orange (Python).

Concentric Circles eps=0.2 | min_samples=5 | k=2 | min_points=5

Method	R	Python	$\lvert\Delta\rvert$
DBSCAN	`0.69009218128208216214`	`0.69009218128208293930`	7.77×10^{-16}
K-Means	`0.02700919079440020423`	`0.02700919079439973586`	4.68×10^{-16}

Two Moons eps=0.2 | min_samples=5 | k=2 | min_points=5

Method	R	Python	$\lvert\Delta\rvert$
DBSCAN	`0.70122811114499916663`	`0.70122811114499850049`	6.66×10^{-16}
K-Means	`0.22303566235762423142`	`0.22303566235762423142`	**0**

Complex9 eps=15.0 | min_samples=5 | k=9 | min_points=5

Method	R	Python	$\lvert\Delta\rvert$
DBSCAN	`0.42094645142453540609`	`0.42094645142454312214`	7.72×10^{-16}
K-Means	`0.05510967752575094553`	`0.05510967752573882328`	1.21×10^{-15}

Complex8 eps=10.0 | min_samples=5 | k=8 | min_points=5

Method	R	Python	$\lvert\Delta\rvert$
DBSCAN	`0.07186876516496862322`	`0.07186876516500724510`	3.86×10^{-15}
K-Means	`0.01225747486413241577`	`0.01225747486413229781`	1.18×10^{-16}

Dartboard1 eps=0.08 | min_samples=5 | k=4 | min_points=5

Method	R	Python	$\lvert\Delta\rvert$
DBSCAN	`0.87434268608030629721`	`0.87434268608031251446`	6.22×10^{-15}
K-Means	`−0.00336367105007065893`	`−0.00336367105007149723`	8.38×10^{-16}

3-Spiral eps=2.0 | min_samples=3 | k=3 | min_points=5

Method	R	Python	$\lvert\Delta\rvert$
DBSCAN	0.5836514046464754690 3	0.5836514046464750249 4	4.44×10^{-16}
K-Means	−0.0014176358526362101 1	−0.0014176358526362020 8	8.03×10^{-18}

Blobs eps=0.5 | min_samples=5 | k=3 | min_points=5

Method	R	Python	$\lvert\Delta\rvert$
DBSCAN	0.8548953162795358551 3	0.8548953162795354110 4	4.44×10^{-16}
K-Means	0.8575691834060311658 0	0.8575691834060308327 3	3.33×10^{-16}

Diagonal Blobs eps=0.8 | min_samples=5 | k=3 | min_points=5

Method	R	Python	$\lvert\Delta\rvert$
DBSCAN	0.7215536253399120747 4	0.7215536253399117416 8	3.33×10^{-16}
K-Means	0.7043253397178761421 3	0.7043253397178755870 2	5.55×10^{-16}

5.5 Interpretation of Differences

Magnitude	Interpretation
$= 0$	Exact bit-for-bit agreement
$< 10^{-14}$	Normal IEEE 754 floating-point rounding; implementations are **numerically equivalent**
$\geq 10^{-12}$	Potential algorithmic discrepancy worth investigating

The largest observed difference across all eight datasets is 6.22×10^{-15} (Dartboard1 DBSCAN), well within the range of normal floating-point accumulation. All differences are at least two orders of magnitude below the DISCO score itself.

5.6 Cross-Dataset Summary

Dataset	DISCO (DBSCAN)	DISCO (K-Means)	K-Means < 0?	Observation
Concentric Circles	0.6901	0.0270	No	K-Means cuts rings
Two Moons	0.7012	0.2230	No	K-Means partially crescents
Complex9	0.4209	0.0551	No	Mixed shapes fav SCAN
Complex8	0.0719	0.0123	No	Density variatio both
Dartboard1	0.8743	-0.0034	**Yes**	4 rings: K-Means harmful
3-Spiral	0.5837	-0.0014	**Yes**	Spiral structure in for K-Means
Blobs	0.8549	0.8576	No	Convex clusters: b rithms succeed
Diagonal Blobs	0.7216	0.7043	No	Elongated but s both work well

Remark 1. *Negative K-Means scores on Dartboard1 and 3-Spiral are* ***correct and expected*** *— K-Means assigns points to clusters that are further away in dc-distance space than the neighbouring cluster.*

For the Blobs and Diagonal Blobs datasets, K-Means achieves a slightly ***higher*** *DISCO score than DBSCAN. This is also expected: when clusters are convex and well-separated (as with Gaussian blobs), centroid-based methods can achieve tight, well-defined partitions that score very well under dc-distance based evaluation.*

6. API Reference

6.1 `disco_score(X, labels, min_points = 5)`

Argument	Type	Description
`X`	matrix	$n \times p$ numeric feature matrix
`labels`	integer vec.	Cluster labels; `-1` for noise
`min_points`	integer	Neighbourhood size ≥ 2. Default: `5`

Returns: Single `numeric` in $[-1, 1]$.

6.2 `disco_samples(X, labels, min_points = 5)`

Returns: `numeric` vector of length n (per-point scores).

6.3 `compute_dc_distances(X, min_points = 5)`

Returns the $n \times n$ DC-distance matrix. Precompute once and reuse:

```
D  <- compute_dc_distances(X, min_points = 5)
s1 <- p_cluster(D, labels1, precomputed_dc_dists = TRUE)
s2 <- p_cluster(D, labels2, precomputed_dc_dists = TRUE)
```

6.4 Utility Functions

Function	Description
`make_circles(n, noise, factor, seed)`	Concentric circles
`make_moons(n, noise, seed)`	Two interleaving crescents
`make_blobs(n, centers, std, seed)`	Gaussian blobs
`plot_disco_scores(X, labels, scores)`	Red–yellow–green scatter plot
`compare_clusterings(X, list(...))`	Rank multiple clusterings by DISCO
`summary_disco_scores(scores)`	Mean, median, SD, quantiles

7. Score Interpretation Guide

Score Range	Label	Meaning
$[0.7,\ 1.0]$	Excellent	Well-separated, compact clusters
$[0.4,\ 0.7)$	Good	Clear cluster structure
$[0.0,\ 0.4)$	Moderate	Overlapping or touching clusters
$[-0.3,\ 0.0)$	Poor	Misassigned points
$[-1.0,\ -0.3)$	Very poor	Clustering worse than random

Common pitfall.

```r
# WRONG: dbscan() returns 0 for noise
labels <- dbscan(X, eps = 0.2)$cluster
# CORRECT: DISCO uses -1 for noise
labels <- dbscan(X, eps = 0.2)$cluster - 1L
```

8. Conclusion

The `disco` R package is a faithful translation of the Python DISCO reference implementation. Three bugs were identified and corrected. Numerical validation across **eight benchmark datasets** — including real-world ARFF datasets and synthetically generated data of varying shapes (rings, crescents, spirals, blobs, diagonal blobs) — confirms that R and Python produce identical results to within normal IEEE 754 floating-point rounding ($< 10^{-14}$).

The results demonstrate that DISCO correctly discriminates between algorithms: DBSCAN outperforms K-Means on non-convex data (rings, spirals), while both methods score comparably on convex Gaussian data (blobs, diagonal blobs). Negative scores on Dartboard1 and 3-Spiral meaningfully penalise K-Means for cutting through genuine cluster structure.

References

[1] Beer, A., Krieger, L., Weber, P., Ritzert, M., Assent, I., Plant, C. (2025). *DISCO: Internal Evaluation of Density-Based Clustering.* arXiv:2503.00127 [cs.LG].

[2] Rousseeuw, P.J. (1987). Silhouettes: A graphical aid to the interpretation and validation of cluster analysis. *Journal of Computational and Applied Mathematics*, 20, 53–65.

[3] Ester, M., Kriegel, H.P., Sander, J., Xu, X. (1996). A density-based algorithm for discovering clusters in large spatial databases with noise. *Proceedings of KDD*, 96(34), 226–231.

[4] Davies, D.L., Bouldin, D.W. (1979). A cluster separation measure. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 1(2), 224–227.

[5] R Core Team (2023). *R: A Language and Environment for Statistical Computing.* R Foundation for Statistical Computing, Vienna, Austria.

[6] Pedregosa, F., et al. (2011). Scikit-learn: Machine Learning in Python. *Journal of Machine Learning Research*, 12, 2825–2830.